# Configure HA Hadoop

## 1) Setting up the hadoop Paths

There are 3 main files that I used to setup the hadoop thing:

### 1) `/etc/enviroment`

has the JAVA_HOME path, we need it as hadoop daemons/processes depend on it and should recognize the path from this file

why `/etc/environment`?

/etc/environment is a SYSTEM-WIDE environment file.

- It applies to ALL users on the system
- It applies to ALL processes and services
- It loads at BOOT TIME before anything else starts
- It is NOT a script — just simple KEY=VALUE pairs

### 2) `/hadoop/etc/hadoop-env.sh`

- It has the needed exports to use hadoop command without typing the full path
- We write the exports + JAVA_HOME + add them to the PATH variable environment

why JAVA_HOME, we already had it in `/etc/environment` ?

- yes in case we do manual running of cluster, so every node has this exports and the they already have JAVA_HOME variable defined already in /etc/environment and it's redundant in this case

- but in automated (like script) running of cluster, We sit on a node and ssh to the other nodes, and because ssh ignores environment variables files because of fast and security reasons. So we need to include them also in hadoop-env.sh

Note: it was recommended in `/hadoop/etc/hadoop-env.sh` to write the exports + in `/etc/profile.d/hadoop-env.sh` or `./bashrc` and just make it contain JAVA_HOME
in order to be recognized on opening new terminal or login respectively

---

## Challenge1:

During SSH, it also ignores ./bashrc and /etc/profile.d/files

I believe that is container thing, this exports should be recognized on opening new terminal or login respectively on linux distros or something like that;

## Solution:

I had to modify the docker-compose.yml at command section to explicitly source the the `/etc/profile.d/hadoop-env` to `./bashrc`

## Note

because of this challenge, in my script I have to put exports or variable at the top of script as ssh will ignores the `/etc/environment` and `/etc/profile.d/hadoop-env.sh` that is sourced in `./bashrc` and hdfs command inherit/export internally the JAVA_HOME

What is the difference between `/etc/profile.d/` and `./bashrc` ?
`./bashrc` is for only on user while `/etc/profile.d/` is for all users

---

### 3) `/etc/profile.d/hadoop-env.sh` and `./bashrc`

`/etc/profile.d/hadoop-env.sh` is the file that has the exports needed (hadoop + ZK exports) and JAVA_HOME path (redundant in case of manual startup of cluster) that is sourced in `./bashrc` in command section in docker-compose.yml

### To Sum Up

- `/etc/environment` ==> has JAVA_HOME that has the path of java JDK and was added to $PATH env variable
- `/hadoop/etc/hadoop-env.sh` ==> has the JAVA_HOME that has the path of java JDK
- `/etc/profile.d/hadoop-env.sh` ==> has the hadoop exports + Zookeeper exports + JAVA_HOME path for java JDK and they were added to $PATH env variable

---

## 2) Setting up the Configuration files

### 1) `hadoop/etc/hadoop/core-site.xml`

core-site.xml is the MAIN configuration file for Hadoop. It contains settings that apply to the ENTIRE Hadoop ecosystem:

- HDFS
- MapReduce
- YARN
- All other components

```
<configuration>
    <!-- tells everyone: use HDFS, NameNode is at hadoop-HA -->
    <property>
        <name>fs.defaultFS</name>
```

```
        <value>hdfs://hadoop-HA</value>
    </property>



    <!-- tells Hadoop where ZooKeeper ensemble is -->
     <property>
        <name>ha.zookeeper.quorum</name>
        <value>master1:2181,master2:2181,worker1:2181</value>
    </property>
</configuration>
```

Note that:
`hdfs://hadoop-HA` this tells that on writing a path like /raw_data/input in hadoop

- the protocol to use ==> `hdfs://`
- and the logical name of cluster that has the NameNode is `hadoop-HA`
  This property tells Hadoop to automatically prefix any relative path with this URI.

Note: if you don't have logical name for your cluster you just put the name of your NameNode and the port to access it
`hdfs://namenode:9000`
│ │ │
│ │ └── Port: NameNode listens here for client connections
│ └──────────── Hostname: the server running the NameNode
└──────────────────────── Protocol: use HDFS (not local file system)

2) `hadoop/etc/hadoop/core-site.xml`

hdfs-site.xml contains settings specific to HDFS only. It configures:

- NameNode behavior
- DataNode behavior
- Replication
- HA settings
- JournalNodes
- Fencing

```
<configuration>

<!-- here we define a logical name for the cluster to use instead of each
service (namenode:9870 , RM:8081, ...etc, that what we used in core-
site.xml)-->
    <property>
        <name>dfs.nameservices</name>
        <value>hadoop-HA</value>
    </property>

<!--logical IDs for my namenodes-->
    <property>
        <name>dfs.ha.namenodes.hadoop-HA</name>
        <value>nn1,nn2</value>
    </property>

<!--where will my nn1 will live in which node and the port that will send
RPC  through-->
    <property>
        <name>dfs.namenode.rpc-address.hadoop-HA.nn1</name>
        <value>master1:8020</value>
    </property>

<!--where will my nn2 will live in which node and the port that will send
RPC  through-->
    <property>
        <name>dfs.namenode.rpc-address.hadoop-HA.nn2</name>
```

```
        <value>master2:8020</value>
    </property>

<!--how we will access nn1 through which port-->
    <property>
        <name>dfs.namenode.http-address.hadoop-HA.nn1</name>
        <value>master1:9870</value>
    </property>

<!--how we will access nn2 through which port-->
    <property>
        <name>dfs.namenode.http-address.hadoop-HA.nn2</name>
        <value>master2:9870</value>
    </property>

<!--where is journal nodes that NameNodes will write to and read from
(8485 is like internal/RPC ports not access/web UI of journal)-->
    <property>
        <name>dfs.namenode.shared.edits.dir</name>

<value>qjournal://master1:8485;master2:8485;worker1:8485/hadoop-
HA</value>
    </property>
```

```
<!--When NameNode fails and client is talking to it: → This class handles
switching the client to the new Active NameNode automatically-->
    <property>
        <name>dfs.client.failover.proxy.provider.hadoop-HA</name>
<value>org.apache.hadoop.hdfs.server.namenode.ha.ConfiguredFailover
ProxyProvider</value>
    </property>
```

```
<!--this class define the fencing mechanism-->
    <property>
        <name>dfs.ha.fencing.methods</name>
        <value>shell(/bin/true)</value>
    </property>


<!--JournalNodes store HDFS edit logs here-->
    <property>
        <name>dfs.journalnode.edits.dir</name>
        <value>/hadoop-3.4.2/dfs/journal</value>
    </property>


<!--
true → ZooKeeper automatically promotes Standby to Active when Active
NameNode dies false → Admin must manually run: hdfs haadmin -failover
nn1 nn2
-->
    <property>
        <name>dfs.ha.automatic-failover.enabled</name>
        <value>true</value>
    </property>


<!--Tells Hadoop where ZooKeeper ensemble is-->
    <property>
        <name>ha.zookeeper.quorum</name>
        <value>master1:2181,master2:2181,worker1:2181</value>
    </property>


<!--
Where NameNode stores its metadata:
→ fsimage (snapshot of file system)
```

```
→ edits (transaction log)
-->
    <property>
        <name>dfs.namenode.name.dir</name>
        <value>/hadoop-3.4.2/dfs/name</value>
    </property>


<!--Where DataNodes store actual HDFS data blocks.-->
    <property>
        <name>dfs.datanode.data.dir</name>
        <value>/datanode_data</value>
    </property>

<!--How many copies of each block to store-->
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
</configuration>
```

## Challenge 2

The automatic failover did not work, as I had sometimes 2 active NameNodes in case of failover, so the suspect was in ZKFC and zookeeper

## Tries

- I tried to use sshfence algorithm as a good fence mechanism that make sure that primary NameNode process is killed not delayed or something like that
- I make sure of configuring SSH Authentication and pass it to the rest of the nodes

- I tried to use like chain of fence mechanism in case of failing one of them , other will be available

## Solution

I end up of using the fake fence algorithm `shell(/bin/true)` and found that there is problem in configuring Zookeeper zoo.cnf file that made the Zookeeper processes in each mode as standalone (not leader and followers); SO I fix it and the fencing works

## 3) `hadoop/etc/hadoop/yarn-site.xml`

YARN = Yet Another Resource Negotiator

- It is the RESOURCE MANAGEMENT layer of Hadoop:
- Manages CPU and RAM across the cluster
- Schedules and runs jobs (MapReduce, Spark etc.)
- Has its own HA setup separate from HDFS HA

```
<configuration>

    <!-- Enabling High Availability for YARN -->
    <property>
        <name>yarn.resourcemanager.ha.enabled</name>
        <value>true</value>
    </property>

     <!--
     Zookeeper can monitor more than one cluster
     Unique name for your YARN cluster.
     Used by ZooKeeper to store state for THIS cluster.
     -->
    <property>
        <name>yarn.resourcemanager.cluster-id</name>
```

```
        <value>hadoop-HA_cluster</value>
    </property>

    <!--logical IDs for resource managers-->
    <property>
        <name>yarn.resourcemanager.ha.rm-ids</name>
        <value>rm1,rm2</value>
    </property>

    <!--node that will have resourse manager 1 (rm1)-->
    <property>
        <name>yarn.resourcemanager.hostname.rm1</name>
        <value>master1</value>
    </property>

    <!--node that will have resourse manager 2 (rm2)-->
    <property>
        <name>yarn.resourcemanager.hostname.rm2</name>
        <value>master2</value>
    </property>

    <!--Web UI port that we will access resource manager 1 from-->
    <property>
        <name>yarn.resourcemanager.webapp.address.rm1</name>
        <value>master1:8088</value>
    </property>

    <!--Web UI port that we will access resource manager 2 from-->
    <property>
        <name>yarn.resourcemanager.webapp.address.rm2</name>
        <value>master2:8088</value>
    </property>

    <!--Tells Hadoop where ZooKeeper ensemble is-->
    <property>
```

```
        <name>hadoop.zk.address</name>
        <value>master1:2181,master2:2181,worker1:2181</value>
    </property>

    <!--if true: ZK automatically promotes Standby RM to Active when
Active RM dies-->
    <property>
        <name>yarn.resourcemanager.ha.automatic-
failover.enabled</name>
        <value>true</value>
    </property>

    <!--
    Tells YARN to store its state in ZooKeeper.
    What state?
    -  Running applications
    - Job progress
    - Resource allocations
    - Container assignments
     -->
    <property>
        <name>yarn.resourcemanager.store.class</name>
<value>org.apache.hadoop.yarn.server.resourcemanager.recovery.ZKRM
StateStore</value>
    </property>
</configuration>
```

---

## 3) Code for Hadoop HA Cluster Initialization & Startup Guide

## Phase 1: Foundation Services

**These services must be running before formatting or starting NameNodes.**

## On Master1, Master2, and Worker1 (My architecture)

### 1. Start ZooKeeper

```
zkServer.sh start
```

**Verify:**

```
jps
```

**Expected process:**

```
QuorumPeerMain
```

---

### 2. Start JournalNodes

```
hdfs --daemon start journalnode
```

**Verify:**

```
jps
```

**Expected process:**

```
JournalNode
```

---

# Phase 2: HDFS HA Initialization

## On Master1 (Active NameNode)

## 1. Format ZooKeeper for HDFS Failover

```
hdfs zkfc -formatZK
```

This initializes the ZooKeeper znodes required for automatic failover.
Perform formatting only once on a fresh cluster (for further info: [Configure HA Zookeeper](Configure HA Zookeeper))

---

## 2. Format the Primary NameNode

```
hdfs namenode -format
```

This creates:

- `VERSION` file
- `fsimage`
- `edits`
  Only execute once on a fresh cluster on the primary NameNode only.

---

## 3. Start the Primary NameNode

```
hdfs --daemon start namenode
```

Verify:

```
jps
```

Expected:

```
NameNode
```

---

## On Master2 (Standby NameNode)

### 1. Bootstrap the Standby NameNode

**This copies metadata from Master1.**

```
hdfs namenode -bootstrapStandby
```

---

### 2. Start the Standby NameNode

```
hdfs --daemon start namenode
```

**Verify:**

```
jps
```

**Expected:**

```
NameNode
```

---

## On BOTH Master1 and Master2

### Start the Failover Controller (ZKFC)

```
hdfs --daemon start zkfc
```

**Verify:**

```
jps
```

**Expected:**

```
DFSZKFailoverController
```

---

## Phase 3: YARN HA Initialization

**YARN maintains its own ZooKeeper-based state store.**

### On Master1

#### 1. Format the YARN State Store

```
yarn resourcemanager -format-state-store
```

⚠️ **Run only once on a fresh cluster.**

---

#### 2. Start the First ResourceManager

```
yarn --daemon start resourcemanager
```

---

### On Master2

#### Start the Second ResourceManager

```
yarn --daemon start resourcemanager
```

**Verify HA State:**

```
yarn rmadmin -getAllServiceState
```

---

## Phase 4: Worker Services

**Now start the DataNodes and NodeManagers.**

### On Worker1, Worker2, Worker3

#### 1. Start DataNodes

```
hdfs --daemon start datanode
```

---

#### 2. Start NodeManagers

```
yarn --daemon start nodemanager
```

**Verify:**

```
jps
```

**Expected:**

```
DataNode
NodeManager
```

# Phase 5: Verification

**Run on Master1.**

---

## 1. Check HDFS HA State

```
hdfs haadmin -getAllServiceState
```

**Expected:**

```
master1 active
master2 standby
```

---

## 2. Check YARN HA State

```
yarn rmadmin -getAllServiceState
```

**Expected:**

```
rm1 active
rm2 standby
```

---

## 3. Check Running Processes

```
jps
```

**On Master1, expected:**

```
NameNode
ResourceManager
DFSZKFailoverController
JournalNode
QuorumPeerMain
```

---

## Complete Formatting Summary (Fresh Cluster Only)

Run these exactly once:

### HDFS Formatting

```
hdfs zkfc -formatZK
hdfs namenode -format
```

### Standby Bootstrap

```
hdfs namenode -bootstrapStandby
```

### YARN Formatting

```
yarn resourcemanager -format-state-store
```

---

## Important Notes

- Never re-run formatting on a running cluster.
- Formatting deletes metadata.
- BootstrapStandby must only be run after the active NameNode is started.

- **ZooKeeper and JournalNodes must be running before HDFS formatting.**

## Output of my automated script

```
PS H:\omar\ITI - Data Management\hadoop-project> docker exec -it master1 bash
root@master1:/# jps
24 Jps
root@master1:/# ./start_cluster.sh
=== Starting ZooKeeper ===
ZooKeeper JMX enabled by default
Using config: /apache-zookeeper-3.8.6-bin/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
ZooKeeper JMX enabled by default
Using config: /apache-zookeeper-3.8.6-bin/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
ZooKeeper JMX enabled by default
Using config: /apache-zookeeper-3.8.6-bin/bin/../conf/zoo.cfg
Starting zookeeper ... STARTED
=== Starting JournalNodes ===
=== Starting NameNodes ===
=== Starting ZKFC ===
=== Starting DataNodes ===
=== Starting YARN ===
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
=== Cluster Status ===
--- master1 ---
291 NameNode
566 ResourceManager
440 DFSZKFailoverController
843 Jps
187 JournalNode
76 QuorumPeerMain
--- master2 ---
656 Jps
272 NameNode
563 ResourceManager
441 DFSZKFailoverController
169 JournalNode
43 QuorumPeerMain
--- worker1 ---
162 JournalNode
611 Jps
298 DataNode
43 QuorumPeerMain
461 NodeManager
--- worker2 ---
200 NodeManager
350 Jps
63 DataNode
--- worker3 ---
200 NodeManager
317 Jps
62 DataNode
=== HA Status ===
master1:8020                                       active
master2:8020                                       standby
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
master1:8033                                       active
master2:8033                                       standby
root@master1:/#
```

```
root@master1:/# jps
14609 JournalNode
14498 QuorumPeerMain
14713 NameNode
15561 Jps
14861 DFSZKFailoverController
14959 ResourceManager
root@master1:/#
```

## Let's test failover and fencing of HDFS

```
root@master2:/# hdfs haadmin -getAllServiceState
master1:8020                                       standby
master2:8020                                       active
root@master2:/# hdfs --daemon stop namenode
WARNING: namenode did not stop gracefully after 5 seconds: Trying to kill with kill -9
ERROR: Unable to kill 13598
root@master2:/# hdfs haadmin -getAllServiceState
master1:8020                                       active
2026-03-04 11:05:56,338 INFO ipc.Client: Retrying connect to server: master2/172.20.0.3:8020. Already tried 0 time(s); retry policy is RetryUpToMaximumCount
WithFixedSleep(maxRetries=1, sleepTime=1000 MILLISECONDS)
master2:8020                                       Failed to connect: Call From master2/172.20.0.3 to master2:8020 failed on connection exception: java.net.
ConnectException: Connection refused; For more details see:  http://wiki.apache.org/hadoop/ConnectionRefused
root@master2:/# hdfs --daemon start namenode
root@master2:/# hdfs haadmin -getAllServiceState
master1:8020                                       active
master2:8020                                       standby
root@master2:/#
```

## let's test failover and fencing of YARN

```
root@master1:/# yarn rmadmin -getAllServiceState
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
master1:8033                               active
master2:8033                               standby
root@master1:/# yarn --daemon stop resourcemanager
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
WARNING: resourcemanager did not stop gracefully after 5 seconds: Trying to kill with kill -9
ERROR: Unable to kill 14959
root@master1:/# yarn rmadmin -getAllServiceState
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
2026-03-04 11:29:07,100 INFO ipc.Client: Retrying connect to server: master1/172.20.0.5:8033. Already tried 0 time(s); retry policy is RetryUpToMaximumCountWithFixedSleep(maxRetries=1, sleepTime=1000 MILLISEC
ONDS)
master1:8033                               Failed to connect: Call From master1/172.20.0.5 to master1:8033 failed on connection exception: java.net.ConnectException: Connection refused; For more detai
ls see:  http://wiki.apache.org/hadoop/ConnectionRefused
master2:8033                               active
root@master1:/# yarn --daemon start resourcemanager
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
root@master1:/# yarn rmadmin -getAllServiceState
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
master1:8033                               standby
master2:8033                               active
root@master1:/#
```